

# Administración de archivos

## Practica01 - Gestor de Contactos

Adiel Jafet Poot Pech

### Reporte: Diferencias entre la clase Buffer y la clase File en Java

En el lenguaje de programación Java, la manipulación de archivos es un aspecto fundamental para el desarrollo de aplicaciones que requieren persistencia de datos o comunicación mediante flujos de entrada y salida. Entre las clases más relevantes en este contexto se encuentran File y las clases relacionadas con Buffer (como BufferedReader, BufferedWriter, BufferedInputStream, entre otras). Aunque ambas se utilizan en el manejo de archivos, cumplen funciones distintas y se aplican en escenarios específicos.

#### Clase File

La clase File, ubicada en java.io, representa una ruta de archivo o directorio en el sistema operativo. Aquí **no se abre ni se lee el archivo en memoria**, solo se consulta al sistema operativo. File **no reserva memoria para contenido del archivo**, solo maneja referencias y metadatos a nivel de sistema.

#### Funcionalidad principal:

- Crear, eliminar y manipular archivos o directorios.
- Consultar propiedades como tamaño, permisos, ruta absoluta o tipo (archivo/directorio).
- Manejar solo la estructura y metadatos, no el contenido.

#### Uso común:

- Verificar si un archivo existe.
- Crear directorios.
- Listar archivos en un directorio.

#### Acceso a memoria:

- No accede directamente al contenido en RAM.
- Consulta al sistema operativo la información del archivo.
- Solo maneja referencias y metadatos a nivel de sistema.

```
File archivo = new File("datos.txt");
if (archivo.exists()) {
    System.out.println("El archivo existe");
}
```

## Clases Buffer (BufferedReader, BufferedWriter, BufferedInputStream, etc.)

Las clases con prefijo Buffered se encuentran en el paquete java.io y son utilizadas para mejorar la eficiencia en la lectura y escritura de datos. Las clases Buffered sí interactúan con la memoria RAM mediante un buffer intermedio. El buffer es un bloque de memoria temporal donde se almacenan datos leídos o que se van a escribir, antes de interactuar con el disco. Esto se hace porque acceder al disco duro es muy costoso en tiempo, mientras que acceder a la RAM es mucho más rápido.

### Funcionalidad principal:

- Almacenamiento temporal en memoria (buffer) que reduce la cantidad de accesos directos al disco.
- Permiten leer o escribir bloques de datos más grandes en una sola operación, en lugar de hacerlo byte por byte o carácter por carácter.

### Ejemplo

#### Lectura con Buffer:

- Cuando lees con BufferedReader, por ejemplo, Java no lee carácter por carácter del disco.
- Lo que hace es:
  1. Leer un bloque grande de datos del archivo y guardarlo en el buffer en RAM.
  2. De ahí, tú vas consumiendo los datos poco a poco (línea por línea o carácter por carácter).
  3. Cuando el buffer se vacía, se vuelve a llenar desde el archivo.

#### Escritura con Buffer:

- Cuando escribes con BufferedWriter, los datos no se escriben inmediatamente en el archivo físico.
- Primero se guardan en el buffer en RAM.
- Cuando el buffer se llena, o cuando llamas a flush() o close(), recién ahí se vacían al disco.

### Ejemplo de escritura:

```
BufferedWriter escritor = new BufferedWriter(new
FileWriter("datos.txt"));
escritor.write("Hola mundo");
// Aún puede estar en RAM, no en el archivo físico
escritor.flush(); // Fuerza a escribir en el archivo
escritor.close();
```

### Diferencias entre File y Buffer

| Aspecto                     | Clase File                                                 | Clases Buffered                                                      |
|-----------------------------|------------------------------------------------------------|----------------------------------------------------------------------|
| <b>Propósito</b>            | Representar un archivo o directorio en el sistema.         | Optimizar la lectura y escritura de datos.                           |
| <b>Manejo de contenido</b>  | No accede directamente al contenido del archivo.           | Sí accede al contenido, pero con almacenamiento intermedio (buffer). |
| <b>Uso principal</b>        | Consultar y manipular propiedades de archivos/directorios. | Mejorar el rendimiento al procesar datos de archivos.                |
| <b>Nivel de abstracción</b> | Más enfocado en el sistema de archivos.                    | Más enfocado en la entrada/salida de datos.                          |

### Recomendaciones de uso

#### Se recomienda usar la clase File cuando:

- Se necesita verificar la existencia de un archivo o directorio.
- Se desea crear, eliminar o renombrar archivos y carpetas.
- Se requiere información sobre permisos, tamaño o rutas.

#### Se recomienda usar clases Buffered cuando:

- Se necesita leer o escribir grandes volúmenes de datos de manera eficiente.
- Se procesan archivos de texto línea por línea.
- Se busca reducir el tiempo de acceso al disco mediante operaciones agrupadas en memoria.